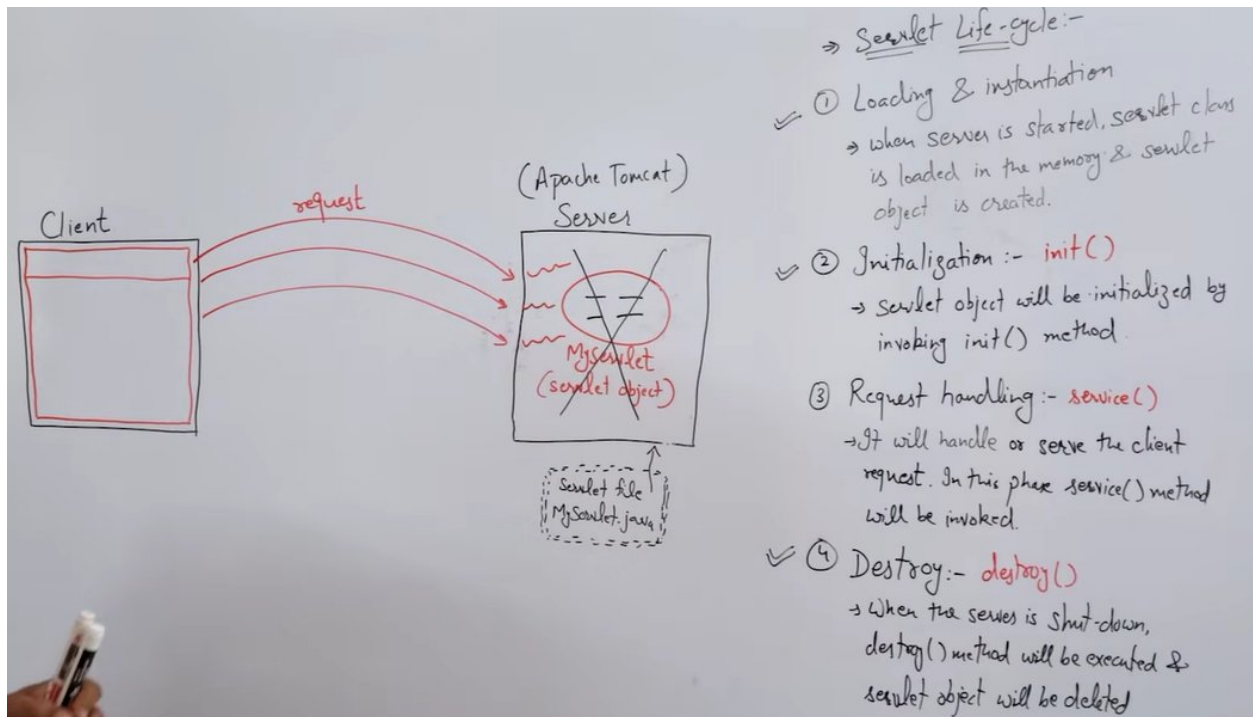




Servlet Life Cycle

Subject: Java / Web Technology

Topic: Servlet Life Cycle

Reference: #4 Life Cycle of Servlet || `init()` `service()` `destroy()` methods of Servlet Hindi

1. Introduction

A **Servlet** is a Java program that runs on a web server and is used to handle requests from clients, such as web browsers.

The **Servlet Life Cycle** describes the different stages a servlet goes through from the time it is created until it is removed from memory.

The servlet life cycle is mainly controlled by the **web container/server**, such as **Apache Tomcat**.

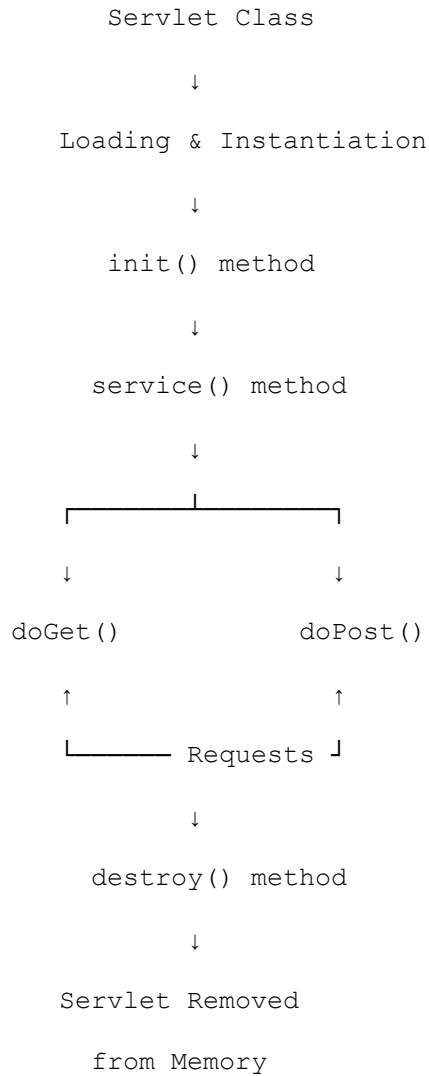
Main Stages

The servlet goes through four major stages:

1. **Loading and Instantiation**
2. **Initialization – `init()`**

3. **Request Handling – `service()`**
4. **Destruction – `destroy()`**

Life Cycle Flow

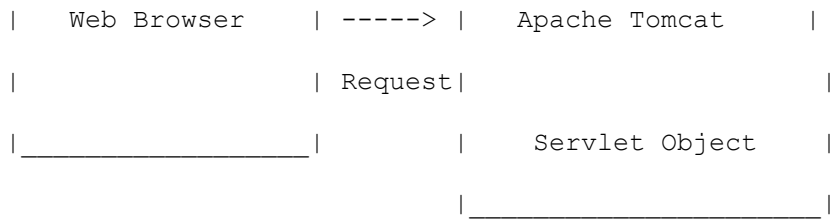


2. Client–Server Interaction

A servlet works between a **client** and a **server**.

Basic Architecture





Explanation

5. The **client**, usually a web browser, sends a request.
6. The request reaches the **web server/container**, such as Tomcat.
7. Tomcat manages the servlet object.
8. The servlet processes the request.
9. A response is sent back to the client.

3. Loading and Instantiation

This is the **first stage** of the servlet life cycle.

What Happens?

When the servlet needs to be used, the web container:

10. **Loads the servlet class** into memory.
11. Creates an **object (instance)** of the servlet.

Example

Suppose we have:

```
public class MyServlet extends HttpServlet {
}
```

The container loads the `MyServlet` class and creates its servlet object.

Execution Frequency

This stage normally occurs **once for a particular servlet instance**.

Important for Exam

Loading: The servlet class is loaded into memory.

Instantiation: An object of the servlet is created.

4. Initialization – `init()`

After the servlet object is created, it must be initialized.

The web container calls the:

```
init()
```

method.

Definition

`init()` Method: The `init()` method is used to perform the initial setup required by a servlet before it starts handling requests.

What Happens?

During initialization, the servlet can:

- Set initial values.
- Read configuration parameters.
- Perform required setup.
- Prepare resources needed by the servlet.

Example

```
public void init() {  
    // Initialization code  
}
```

Execution Frequency

The `init()` method is called **only once** during the life cycle of a servlet instance.

Important for Exam

`init()` is called once before the servlet starts processing requests.

5. Request Handling – `service()`

The `service()` method is the most active stage of the servlet life cycle.

It is responsible for handling client requests.

What Happens?

Whenever a client sends a request to the servlet:

Client Request → *service()*

The container invokes the `service()` method.

Execution Frequency

Unlike `init()`, the `service()` method can be called **many times**.

It is called whenever a new request is received.

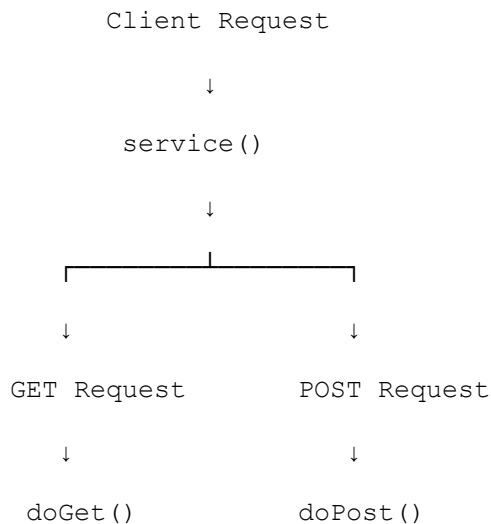
6. Role of `service()` Method

The `service()` method determines what type of HTTP request has been received and delegates it to the appropriate method.

The common methods are:

- `doGet()`
- `doPost()`

Request Flow



GET Request

If the request is a **GET** request:

`service()` → `doGet()`

POST Request

If the request is a **POST** request:

`service()` → `doPost()`

Important for Exam

The `service()` method acts as a **request dispatcher** for HTTP methods.

7. Multithreading in Servlet

One important feature of servlet request handling is **multithreading**.

What is Multithreading?

Multithreading means multiple tasks can be handled concurrently using different threads.

When multiple clients send requests, the servlet container can use different threads to process those requests.

Conceptual Example

Client 1 — Request —> Thread 1 —> Servlet

Client 2 — Request —> Thread 2 —> Servlet

Client 3 — Request —> Thread 3 —> Servlet

This allows a servlet to handle multiple requests efficiently.

Important Point

The servlet object is generally created once, while **multiple request-processing threads** may use that servlet instance.

Therefore, servlet code should be designed carefully when accessing shared data.

8. Destruction — `destroy()`

The final stage of the servlet life cycle is **destruction**.

The web container calls:

```
destroy()
```

when the servlet is being taken out of service.

What Happens?

The `destroy()` method is used for **cleanup operations**.

For example, it may be used to:

- Close database connections.
- Close files.
- Release resources.
- Perform other cleanup operations.

Example

```
public void destroy() {  
    // Cleanup code  
}
```

Execution Frequency

The `destroy()` method is called **once** for a servlet instance before it is removed.

9. Servlet Object Removal

After the `destroy()` method has completed, the servlet is no longer needed.

The servlet object can then be removed from memory by the Java runtime's **garbage collection mechanism**.

Life Cycle

Servlet Object Created

↓

`init()`

↓

`service()`

↓

Multiple Requests

↓

`destroy()`

↓

Object Becomes Eligible

for Garbage Collection

Important Point

`destroy()` performs cleanup; it does not itself directly perform Java memory deallocation.

10. Complete Servlet Life Cycle

The complete process can be understood as follows:

Step 1: Loading

The servlet class is loaded into memory.

↓

Step 2: Instantiation

The servlet object is created.

↓

Step 3: Initialization

The container calls:

```
init()
```

↓

Step 4: Request Processing

For each incoming request, the container calls:

```
service()
```

↓

Step 5: Request Delegation

Depending on the HTTP request:

```
GET → doGet()
```

```
POST → doPost()
```

↓

Step 6: Destruction

When the servlet is taken out of service, the container calls:

```
destroy()
```


↓

Step 7: Removal

The servlet object becomes eligible for garbage collection.

11. Execution Frequency of Life-Cycle Methods

Stage / Method	Purpose	Frequency
Loading	Loads servlet class	Generally once per servlet instance
Instantiation	Creates servlet object	Generally once
init()	Initializes servlet	Once
service()	Handles requests	Repeatedly
doGet()	Handles GET requests	As required
doPost()	Handles POST requests	As required
destroy()	Performs cleanup	Once

Easy Way to Remember

init() → Once

service() → Many times

destroy() → Once

Once → Many → Once

12. init() VS service() VS destroy()

Feature	init()	service()	destroy()
Purpose	Initialization	Request processing	Cleanup
Called	Before request processing	For incoming requests	When taken out of service
Frequency	Once	Multiple times	Once
Main Work	Setup	Handle requests	Release resources
Example	Initialize configuration	Process GET/POST	Close resources

13. Important Exam Points

Important for Exam:

- The **Servlet Life Cycle** is controlled by the **web container**.
- **Apache Tomcat** is an example of a servlet container.
- The servlet class is loaded before the servlet object is created.
- The `init()` method is used for **initialization**.
- `init()` is called **once** for a servlet instance.
- The `service()` method handles client requests.
- `service()` can be called **multiple times**.
- `service()` delegates requests such as GET and POST to methods like `doGet()` and `doPost()`.
- Servlet containers use **threads** to handle requests concurrently.
- The `destroy()` method is used for **cleanup**.
- `destroy()` is called **once** before the servlet is taken out of service.
- After destruction, the servlet object can become eligible for **garbage collection**.

14. Commonly Confused Concepts

`init()` VS `service()`

`init()`

- Used for initialization.
- Called once.
- Runs before request processing.

`service()`

- Used for handling requests.
- Can run many times.
- Called whenever requests are processed.

`service()` VS `doGet()` / `doPost()`

`service()` is responsible for receiving the request and determining which request-handling method should be used.

`service()`

|

| — GET → `doGet()`

|
└─ POST → doPost()

destroy() vs Garbage Collection

destroy()

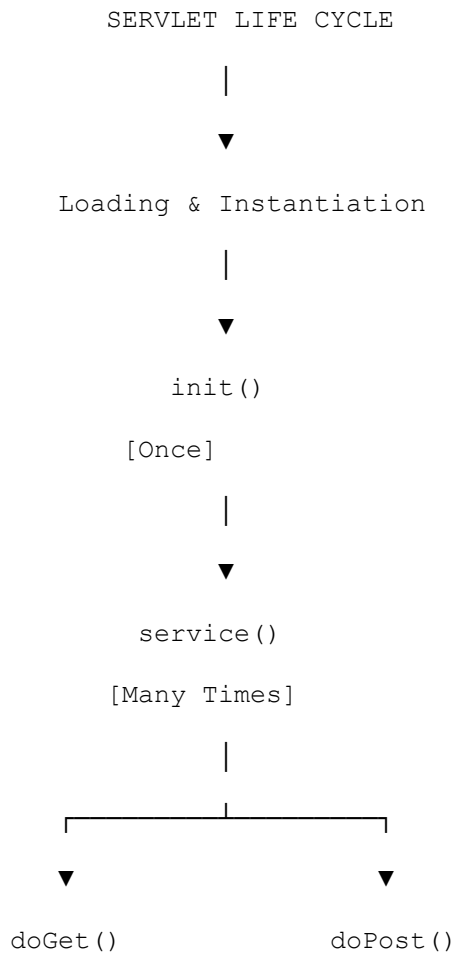
- Performs servlet cleanup.
- Called by the container.

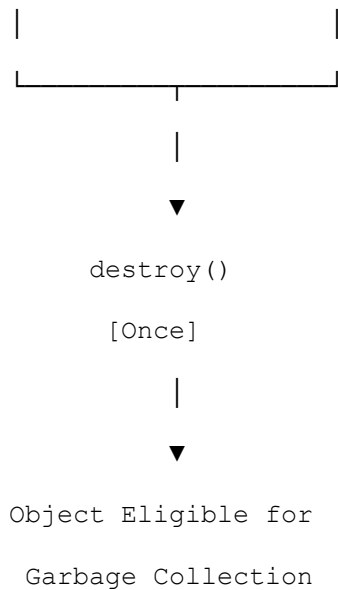
Garbage Collection

- Reclaims memory from objects that are no longer reachable.
- Managed by the Java Virtual Machine.

15. Quick Revision

Servlet Life Cycle in One Diagram





16. Key Points

- **Servlet Life Cycle** describes the complete life of a servlet from creation to removal.
- The servlet container manages the life cycle.
- **Loading and instantiation** create the servlet instance.
- `init()` performs initialization and is called once.
- `service()` handles client requests and may be called many times.
- `doGet()` handles GET requests.
- `doPost()` handles POST requests.
- Servlet request processing commonly uses **multiple threads**.
- `destroy()` performs cleanup and is called once.
- After destruction, the object may become eligible for **garbage collection**.

17. Summary

The **Servlet Life Cycle** consists of four major stages: **loading and instantiation, initialization, request processing, and destruction**.

First, the servlet class is loaded and its object is created. The container then calls `init()` once to initialize the servlet. Whenever clients send requests, the container calls `service()`, which handles the requests and delegates them to methods such as `doGet()` and `doPost()`. Finally, when the servlet is taken out of service, the container calls `destroy()` to perform cleanup.

One-Line Revision

<i>Load → Create → init() → service() → destroy() → Garbage Collection</i>
--

Most important sequence to memorize:

init() → service() → destroy()

Frequency:

Once → ManyTimes → Once

Video Link: https://youtu.be/6Rhdx-d_pbY?si=E21cxXfls4pUoCMn

Vidoe Title: #4 Life Cycle of Servlet || init() service() destroy() methods of Servlet Hindi